

System Design and Development

The application was developed in Java with a MySQL database and uses a layered structure consisting of the application logic, model classes, and Data Access Objects (DAOs). The model classes represent the main objects in the system, such as Car, Reservation, Customer, and Invoice. The DAO classes are responsible for communicating with the MySQL database and handling operations such as creating, retrieving, and updating records. The main application handles user interaction through a text-based interface. When a user performs an operation such as creating or returning a reservation, the application uses the model classes to perform the necessary business logic and then uses the appropriate DAO to save changes to the database. When information is needed from the database, the DAO retrieves the data and creates the corresponding Java objects. I also used design patterns in several parts of the application. I used the Singleton design pattern for the list of cars so that the application could maintain a single shared collection of cars. The list is initially populated from the database and can then be accessed throughout the application without repeatedly querying the database for the same car information. This reduces unnecessary database calls and keeps the application's car state consistent. I also used the Strategy design pattern for handling different fuel types. Different fuel types have different refueling costs, so instead of putting all of the fuel pricing logic into the Car class, I created separate fuel behavior classes such as GasBehavior, DieselBehavior, HybridBehavior, and ElectricBehavior. The car selects the appropriate behavior based on its fuel type, allowing the refueling calculation to vary without requiring the Car class to contain separate logic for every fuel type. The application was made as a text-based application because the main focus of the assignment was the functionality of the rental system rather than creating a GUI.

Testing

For testing, I used JUnit to test the different parts of the application. I mainly used it to test the logic within my classes and make sure that methods returned the expected results when given different inputs. I tested different scenarios such as calculating fees, checking fuel levels, checking mileage, and creating and returning reservations.

What Went Wrong

The main problem I ran into was with testing my DAO classes using JUnit. With how I originally implemented my DAO classes, they directly obtained a database connection and interacted with the database. This meant that when I was testing a DAO method, it could actually create or change an entry in my database. This made testing the DAO classes less efficient because I had to be careful about what data was being added to the database, and I could not easily run the same tests repeatedly without affecting the database. Because of this, I was not able to isolate the database-related functions as effectively as I would have liked. The rest of the application logic could be tested independently with JUnit, but the DAO testing was more dependent on the actual database.

What I Learned

One of the biggest things I learned from this project was how important it is to think about testing when designing classes, instead of trying to add testing after the classes have already been created. In the future, when creating a DAO-related class, I would structure it so that the database connection could be passed into the DAO as a parameter instead of having the DAO create its own connection. This would make it possible to use Mockito to mock the database connection when running JUnit tests. This would allow me to test database-related functions without actually changing the database every time a test is run. It would also make the testing process faster and make it easier to create a larger number of isolated tests for the DAO classes.

Another thing I learned is that when I pull objects from the database, I should keep the database ID with the object in Java. This allows me to use the ID when I need

to update, delete, or find that specific object in the database instead of having to search for it using other information about the object. Using a unique database ID also makes these database operations more reliable as the amount of data in the database increases.

Technology Stack

- Java - Main programming language used to implement the application.
- MySQL - Database used to store cars, customers, and reservations.
- Maven - Used to manage the project and its dependencies.
- JUnit - Used for unit testing the application logic.
- JDBC - Used to connect the Java application to the MySQL database.